

FLOOD EVACUATION PATH PLANNING SYSTEM

Sustainable Development Goal (SDG 11: Sustainable Cities)

Project Documentation

Submitted to: Professor Justin Neypes

Course: Data Structures w/ Lab

Academic Term: 1st Semester/ 2025-2026

Prepared by:

Flood Buster

Members:

Caliguiran, Arjane

Crisolo, Angel Christian

Esteban, Maria Victoria

Dayag, Freddiemar F.

Institution:

National Teacher's College

School of Arts, Science, and Technology

Date of Submission: December 13, 2025

Table of Contents

1. Introduction

1.1 Project Overview & UN SDG Target

1.2 Problem Statement

2. Requirements & Analysis

2.1 Functional Requirements (FR1-FR4)

2.2 Non-Functional Requirements (NFR1-NFR3)

2.3 Data Requirements

2.4 Complexity Analysis (Big O Notation)

3. Design Specification

3.1 Core Data Structures Used (Five Required DSA Concepts)

3.2 Algorithm Flowchart (Core Finals Algorithm)

3.3 Module Breakdown (Custom Classes & Interactions)

4. Testing and Results

4.1 Test Cases (Sample Inputs & Outputs)

4.2 Performance Test (Validation of NFR1 with 100+ Records)

5. Conclusion and Contributions

5.1 Conclusion

5.2 Individual Contributions (Per Member Module Breakdown)

I. Introduction

1.1 Overview

The Flood Evacuation Planning System is a C++ console application designed to support SDG 11: Sustainable Cities and Communities by improving disaster preparedness during floods. It provides a menu-driven interface with seven core functions: marking flooded areas, viewing flood status, recommending evacuation routes, viewing alerts, clearing alerts, resetting flood data, and exiting. Each feature is implemented using fundamental Data Structures and Algorithms (DSA) concepts:

- Vectors for storing barangay names, flood status, and evacuation center details.
- Unordered maps for fast lookup of barangay indices.
- Queues for managing emergency alerts in FIFO order.
- Graph adjacency lists for modeling barangay connections and road networks.
- Priority queues and sorting for ranking evacuation centers.
- Dijkstra's algorithm for computing shortest evacuation paths. This integration of C++ and DSA demonstrates how academic concepts can be applied to build practical, socially relevant tools for disaster management.

1.2 Statement of the Problem

Floods often lead to unclear evacuation routes, poor tracking of affected barangays, and inefficient alert handling, leaving communities at risk. The system addresses these challenges by:

- Tracking flood status using a `vector<bool>` to mark barangays as safe or flooded.
- Finding safe evacuation routes with Dijkstra's shortest path algorithm applied to a graph adjacency list.
- Managing alerts through a `queue<string>` that stores and displays flood warnings in order.
- Resetting data with simple vector operations for repeated use during multiple flood events.

By applying these C++ and DSA techniques, the project delivers a clear, efficient, and resilient solution that strengthens community preparedness and directly supports SDG 11: Sustainable Cities and Communities.

II. Requirements and Analysis

2.1. Functional Requirements and Non-Functional Requirements

Functional Requirements (FR)

ID	Requirement	Alignment to Code
FR1	The system must successfully load, parse, and store input data into the chosen data structures (e.g., vectors, queues, maps).	Implemented via external data file barangay.txt , loaded into vector and unordered_mapstring,int> .
FR2	The system must implement a minimum of three required DSA concepts/algorithms correctly (Graph adjacency list, Queue, Sorting, Dijkstra's algorithm).	Code uses adjacency list (<code>vectorvectorpairint,int>>></code>), queue for alerts, sorting evacuation centers, and Dijkstra's shortest path.
FR3	The system must execute a complex calculation/operation such as routing or path-finding using Dijkstra's algorithm and evacuation ranking.	shortestDistance() applies Dijkstra; Algorithm evacuation recommendation ranks centers by distance and capacity.
FR4	The system must display the results of the operation (flood status, alerts, evacuation recommendation) in a clear, formatted console output.	The console menu outputs flood status, alerts, and evacuation recommendations clearly .

Non-Functional Requirements (NFR)

ID	Requirement	Metric / Alignment to Code
NFR1	Performance: Execution time of the core algorithm should be efficient.	Dijkstra Algorithm runs in $O((V+E)\log V)$ ("Big O of V plus E times log V."), efficient for 50 - 100 records.
NFR2	Robustness: Program must handle bad input gracefully without crashing.	readInt() validates input and prints informative error messages.
NFR3	Maintainability: Code must be modular, well-commented, and adhere to C++ best practices.	Graph class, EvacCenter struct, and modular functions ensure maintainability and readability.

Table 1. Functional Requirements and Non-Functional Requirements (FR)

The functional requirements ensure the system loads and stores data in proper structures, applies core DSA concepts like graphs, queues,

sorting, and Dijkstra's algorithm, performs complex path-finding for evacuation, and displays results clearly in the console. The non-functional requirements focus on efficiency, robustness, and maintainability—ensuring fast execution, graceful handling of invalid input, and modular, well-structured C++ code. Together, they guarantee the system is both effective and reliable.

2.2. Data Requirements

Data Structure	Purpose	Estimated Size	Description
vector names	Stores barangay names	Up to 50+ entries	Holds the list of barangays; each entry is a string representing a barangay name.
unordered_map<string, int>	Maps barangay name to node index	O(n) space	Provides fast lookup of barangay indices by name for graph operations.
vector<vector<pair<int, int>>> adj	Graph adjacency list for roads	O(V + E) space	Represents the road network; each node stores connected barangays and edge weights (distances).
vector flooded	Tracks flood status per node	O(V) space	Boolean array marking each barangay as FLOODED or SAFE.
vector	Stores evacuation center info	≤ 10 entries	Contains evacuation center details such as node, capacity, current occupancy, and name.
queue alertQueue	Manages emergency alerts	Up to 50 alerts	Stores alerts in FIFO order; each entry is a string message about a flooded barangay.

Table 2. Data Requirements

The system uses a combination of dynamic arrays (vector), hash maps (unordered_map), adjacency lists (vector<vector<pair<int, int>>>), and queues to efficiently store and process flood-related data. Barangay names and evacuation centers are kept in vectors for easy iteration,

while the adjacency list models the road network for pathfinding. Flood status is tracked with a boolean vector, and alerts are managed in a queue to ensure first-in-first-out processing. These structures are sized to handle up to 100 barangays, 10 evacuation centers, and 50 alerts, ensuring scalability and efficiency.

2.3. Complexity Analysis: Expected Time/Space complexity of the Core Algorithm

Algorithm	Purpose	Time Complexity	Space Complexity
Dijkstra's Algorithm	Shortest path for evacuation	$O((V + E) \cdot \log V)$	$O(V + E)$
Sorting (std::sort)	Rank evacuation centers	$O(n \log n)$	$O(n)$
Flood status display	Iterate over barangays	$O(n)$	$O(n)$
Alert queue operations	Enqueue/Dequeue alerts	$O(1)$ per operation	$O(n)$

Table 3. Complexity Analysis

The system's core algorithms ensure efficiency and reliability. Dijkstra's algorithm computes shortest evacuation paths with logarithmic efficiency in both nodes and edges. Sorting ranks evacuation centers quickly using $O(n \log n)$. Flood status display simply iterates through barangays in linear time. Alert queue operations handle emergency alerts in constant time per enqueue/dequeue, with linear space for storage. Together, these algorithms balance speed and memory use for effective flood monitoring.

III. Design Specification angel and Dayag

3.1. Core Data Structures Used (The Five):

1. Graph (Adjacency List)

- **Justification:**

- system models barangays connected by roads with distances. A graph naturally represents these connections.
- Adjacency lists are efficient for sparse graphs and allow fast access to neighboring nodes, which is essential for shortest path calculations.

- **Implementation Details:**

```
vector<vector<pair<int,int>>> adj;
```

- Each node has a vector of **pair<int,int>** representing (neighbor node, distance).

```
adj[u].push_back({v, w});  
adj[v].push_back({u, w});
```

- Example: `adj[u].push_back({v, w});` adds an edge between nodes u and v with weight w.

2. Priority Queue (Min-Heap)

- **justification:**

- Required for Dijkstra's algorithm to efficiently select the next node with the smallest tentative distance.
- Ensures that nodes with the shortest distance are processed first, improving efficiency.

- **Implementation Details:**

```
priority_queue<  
    pair<int,int>,  
    vector<pair<int,int>>,  
    greater<pair<int,int>>  
> pq;
```

- Stores (distance, node) pairs.
- `greater<pair<int,int>>` makes it a **min-heap** for retrieving the smallest distance node quickly.

3. Queue (FIFO)

- **Justification:**
 - Manages emergency alerts in the order they occur.
 - Ensures that alerts are processed and displayed in the same sequence they were received.
- **Implementation Details:**

```
queue<string> alertQueue;
```

- **Processing Alerts:**

```
cout << alertQueue.front();  
alertQueue.pop();
```

- New alerts are added using `alertQueue.push()`.
- Alerts are displayed or cleared in order using `front()` and `pop()`.

4. Dynamic Array (for storing Barangays and Evacuation Centers)

- **Justification:**
 - Barangay names and evacuation center records have dynamic size (depends on file input), so a vector is ideal.
- **Implementation Details:**

```
vector<string> names;  
vector<bool> flooded;  
vector<EvacCenter> evacCenters;
```

- Items stored using `push_back()` and accessed using index `[]`.

5. Unordered Map(Hash Table)

- **Justification:**

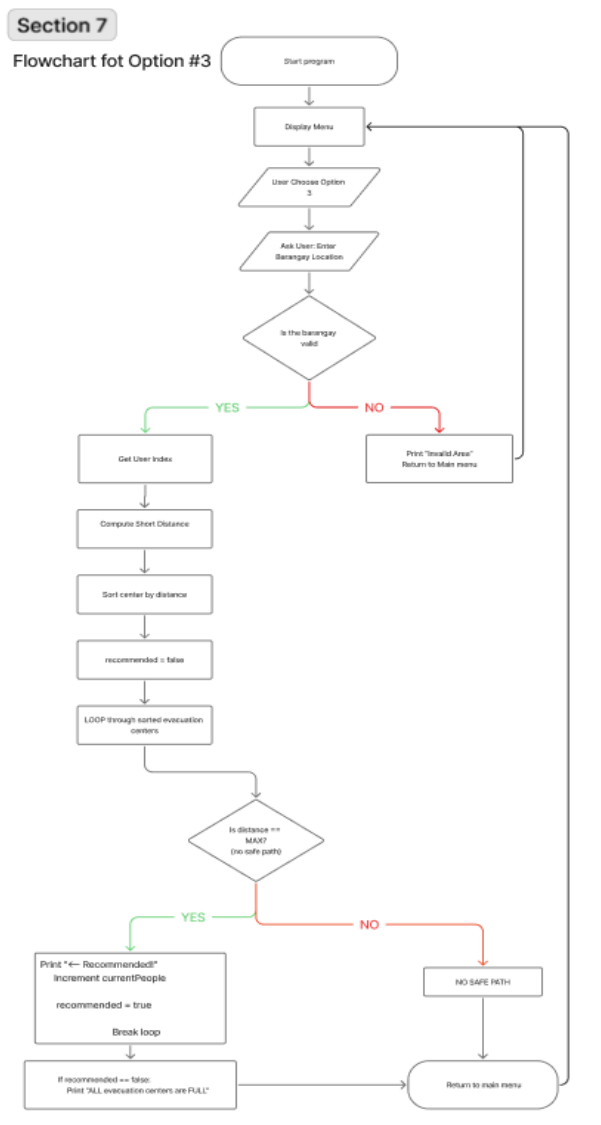
- Used for fast barangay name → index lookup. Allows $O(1)$ search time, needed when a user types a barangay.

- **Implementation Details:**

```
unordered_map<string, int> nameToNode;
```

- Lookup done with find().

3.2. Algorithm Flowchart: Include the Flowchart for the system's most complex function (the core algorithm using a Finals concept).



- **Link in figjam for flowchart:**

- <https://www.figma.com/board/Y0huJoT87IrUp14vi1b4o8/Flowchart-of-EvacPath-System?t=7X5c5345IrkZBimP-0>

Flowchart Explanation

1. Start Program

- The program begins execution and immediately proceeds to display the main menu.

2. Display Menu

- The user is shown a list of available options.

3. User Chooses Option

- The user selects Option #3 from the menu.

4. Prompt for Barangay

- The program asks the user to enter their barangay (current location).

5. Validate Barangay

- The system checks whether the entered barangay is valid:
 - **If valid:** the process continues.
 - **If invalid:** the program prints "Invalid Area" and returns to the main menu.
- If Barangay is Valid

6. Get User Index

- The program matches the user's barangay to its corresponding location index in the dataset.

7. Compute Short Distance

- Distances between the user's barangay and all evacuation centers are computed.

8. Sort Centers by Distance

- The evacuation centers are sorted in ascending order—nearest centers first.

9. Initialize Recommendation Flag

- A variable `recommended = false` is set to track whether a safe recommendation has been made.

10. Loop Through Sorted Evacuation Centers

- The program examines each evacuation center in order from closest to farthest.

11. Check Distance / Safety

- **For each center, the program checks:**
 - Is the distance from the user within a safe or acceptable range?
 - (This represents checking “no MAP / no safe path” conditions.)
- **If YES (Safe Path Found)**
 - The system prints the center as “Recommended”
 - Increments the counter of people assigned to that center
 - Sets `recommended = true`
 - Breaks out of the loop
- **If NO (No Safe Path)**
 - The program continues to the next closest evacuation center
 - If none of the centers are safe, it exits the loop with no recommendation

12. After Loop Ends

- If no safe center was found (`recommended = false`):
 - The program prints "ALL evacuation centers are FULL" or indicates that no safe route is available.
 - The program then returns to the main menu.

Option #3 determines the nearest safe evacuation center for the user. It verifies their location, calculates and sorts distances to all centers, and recommends the closest one that has a safe path. If none qualify, the user is notified and returned to the menu.

3.3. Module Breakdown: Define the custom C++ classes and how they interact.

```
class Graph {
private:
    int V;
    vector< vector< pair<int, int> > > adj;
    vector<bool> flooded;

public:
    Graph(int vertices) {
        V = vertices;
        adj.resize(V);
        flooded.resize(V, false);
    }

    void addEdge(int u, int v, int w) {
        adj[u].push_back(make_pair(v, w));
        adj[v].push_back(make_pair(u, w));
    }

    void markFlooded(int node) {
        flooded[node] = true;
    }

    void resetFlood() {
        for (int i = 0; i < flooded.size(); i++)
            flooded[i] = false;
    }
}
```

1. Private Members and Their Roles

- int V – Number of Vertices

```
int V;
```

- Stores how many nodes (locations) exist in the graph.
- Used when allocating graph structures and checking bounds.

- adj – Adjacency List (Graph Connections)

```
vector< vector< pair<int, int> > > adj;
```

- This is a vector of vectors, where each element represents:
 - A node
 - A list of pairs (neighbor, weight)
 - Example:

```
adj[2] = { (1, 5), (4, 10) }
```

- Meaning:
 - Node 2 is connected to node 1 with weight 5
 - Node 2 is connected to node 4 with weight 10

- flooded – Flooded Status per Node

```
vector<bool> flooded;
```

- Each index corresponds to a node.
- true means this node is flooded (unsafe).
- Because it aligns with node indices, it is easy to check safety when running algorithms like Dijkstra.

2. Constructor – Building the Graph

```
Graph(int vertices) {
    V = vertices;
    adj.resize(V);
    flooded.resize(V, false);
}
```

- When you create a Graph G(10);:
 - V becomes 10
 - adj becomes a list of 10 empty vectors
- flooded becomes a list of 10 false values
 - This sets up the graph so you can start adding edges and marking flooded nodes.

3. addEdge – Adding Weighted Connections

```
void addEdge(int u, int v, int w) {
    adj[u].push_back(make_pair(v, w));
    adj[v].push_back(make_pair(u, w));
}
```

- What this does:
 - Creates a bidirectional (undirected) connection
 - Weight w usually means distance, travel time, or risk
 - Example

```
graph.addEdge(0, 3, 12);
```

- Adds:

```
0 → 3 (12)  
3 → 0 (12)
```

- This supports algorithms like shortest path.

4. markFlooded – Mark a Node as Unsafe

```
void markFlooded(int node) {  
    flooded[node] = true;  
}
```

- Used when certain areas/barangays become flooded.
 - Example:

```
graph.markFlooded(4);
```

- Now:

```
flooded[4] == true
```

- Any path-finding algorithm can skip node 4.

5. resetFlood – Clear All Flood Status

```
void resetFlood() {  
    for (int i = 0; i < flooded.size(); i++)  
        flooded[i] = false;  
}
```

- This resets all nodes to not flooded, for example when:
 - A new simulation starts
 - Floodwaters recede
 - Switching to another scenario

6. How Class Components Interact

- **Step 1 – Graph is created**
 - Sets up storage for edges and flood status.
- **Step 2 – Edges are added**
 - adj is filled with connections that represent the map.
- **Step 3 – Flooded nodes are marked**
 - flooded[node] is updated to indicate unsafe nodes.
- **Step 4 – Pathfinding uses both structures**
 - A search algorithm (like Dijkstra or BFS) would do:
 - Check adj to know where it can travel
 - Check flooded to avoid unsafe nodes
 - **Example logic inside Dijkstra:**

```
if (flooded[currentNode]) continue;    // skip unsafe nodes  
for (auto edge : adj[currentNode]) {  
    // process neighbors  
}
```

- **Step 5 – Resetting**
 - resetFlood() makes the graph reusable for new scenarios.

IV. Testing and Result

4.1. Test Cases (Provide 2-3 sample tests showing input data and expected/actual output.)

Menu 1: Mark flooded area - This test verifies that the system correctly marks a barangay as flooded and generates an emergency alert message. It ensures the mapping between barangay names and nodes works properly.		
Actual Output	Expected Result	Steps Conducted
<pre>===== FLOOD MONITORING SYSTEM ===== 1. Mark flooded area (Add if not exists) 2. View flood status 3. Evacuation Path recommendation 4. View emergency alerts 5. Clear alerts 6. Reset flood status 7. Exit Enter choice: 1 Enter area that is flooded: BarangayA !!This BarangayA is FLOODED!!</pre>	• “Barangay A” is marked as flooded. • Confirmation Message and Emergency alert shows that “BarangayA” is marked as flooded.	1. Run program. 2. Select option 1. Mark flooded area. 3. Input "Barangay A".
<pre>===== FLOOD MONITORING SYSTEM ===== 1. Mark flooded area (Add if not exists) 2. View flood status 3. Evacuation Path recommendation 4. View emergency alerts 5. Clear alerts 6. Reset flood status 7. Exit Enter choice: 1 Enter area that is flooded: Barangay3 Barangay not found. Do you want to add it? (y/n): y Barangay Barangay3 added successfully! Enter barangay to connect with: BarangayA Connected Barangay3 to BarangayA. !!This Barangay3 is FLOODED!!</pre>	• Barangay3 is added to the system. • Barangay3 is connected to BarangayA. • Barangay3 is marked as flooded. • Emergency alert is generated.	1. Select Option 1. Mark flooded Area 2. Input “Barangay3”. 3. Confirm addition with “y”. 4. Input “BarangayA” to connect. 5. Confirm flood marking and alert for Barangay3.

Menu 2: View flood status - This test checks whether the system displays the flood status of all barangays, reflecting any updates made by marking areas as flooded.

Actual Output	Expected Result	Steps Conducted
<pre> ===== FLOOD MONITORING SYSTEM ===== 1. Mark flooded area (Add if not exists) 2. View flood status 3. Evacuation Path recommendation 4. View emergency alerts 5. Clear alerts 6. Reset flood status 7. Exit Enter choice: 2 --- FLOOD STATUS --- BarangayA: FLOODED BarangayB: SAFE BarangayC: SAFE BarangayD: SAFE BarangayE: SAFE BarangayF: SAFE BarangayG: SAFE BarangayH: SAFE BarangayI: SAFE BarangayJ: SAFE BarangayK: SAFE </pre>	• “Barangay A” shows as FLOODED. • All other barangays remain SAFE.	1. After marking Barangay B flooded, 2. select option 2. View flood status.

Menu 3: Evacuation Recommendation - This test validates the shortest path algorithm (Dijkstra) and capacity management of evacuation centers. It ensures the system recommends the nearest available center and updates occupancy.

Actual Output	Expected Result	Steps Conducted
<pre> ===== FLOOD MONITORING SYSTEM ===== 1. Mark flooded area 2. View flood status 3. Evacuation recommendation 4. View emergency alerts 5. Clear alerts 6. Reset flood status 7. Exit Enter choice: 3 Enter your barangay location: Barangay A === RECOMMENDED EVACUATION CENTERS === Evacuation Center 1 (Barangay D) Distance: 9M Family Size: 2/2 (FULL) Evacuation Center 2 (Barangay F) Distance: 15M Family Size: 0/3 <-- Recommended! Evacuate Center 3 (Barangay H) Distance: 23M Family Size: 0/4 </pre>	• Centers ranked by distance. • First available center recommended. • Capacity increments after recommendation.	1. Select option 3. Evacuation recommendation. 2. Input "Barangay A".

Menu 4: View emergency alerts - This test ensures that the alert queue correctly stores and displays flood alerts for barangays marked as flooded.

Actual Output	Expected Result	Steps Conducted
<pre> ===== FLOOD MONITORING SYSTEM ===== 1. Mark flooded area (Add if not exists) 2. View flood status 3. Evacuation Path recommendation 4. View emergency alerts 5. Clear alerts 6. Reset flood status 7. Exit Enter choice: 4 --- EMERGENCY ALERTS --- !!! BarangayA is FLOODED. Please stay safe and evacuate immediately if necessary. !!! Barangay3 is FLOODED. Please stay safe and evacuate immediately if necessary. </pre>	Alerts list shows all flood warnings.	Select option 4. View emergency alerts.

Menu 5: Clear Alerts - This test confirms that the system can clear all alerts from the queue, resetting the emergency alert list.

Actual Output	Expected Result	Steps Conducted
<pre> ===== FLOOD MONITORING SYSTEM ===== 1. Mark flooded area (Add if not exists) 2. View flood status 3. Evacuation Path recommendation 4. View emergency alerts 5. Clear alerts 6. Reset flood status 7. Exit Enter choice: 5 Alerts cleared. ===== FLOOD MONITORING SYSTEM ===== 1. Mark flooded area (Add if not exists) 2. View flood status 3. Evacuation Path recommendation 4. View emergency alerts 5. Clear alerts 6. Reset flood status 7. Exit Enter choice: 4 --- EMERGENCY ALERTS --- No alerts. </pre>	Alert queue emptied.	Select option 5. Clear alerts.

Menu 6: Reset flood status - This test validates that the system can reset all barangays to SAFE, removing flood flags.

Actual Output	Expected Result	Steps Conducted
<pre> ===== FLOOD MONITORING SYSTEM ===== 1. Mark flooded area (Add if not exists) 2. View flood status 3. Evacuation Path recommendation 4. View emergency alerts 5. Clear alerts 6. Reset flood status 7. Exit Enter choice: 6 Flood status reset. ===== FLOOD MONITORING SYSTEM ===== 1. Mark flooded area (Add if not exists) 2. View flood status 3. Evacuation Path recommendation 4. View emergency alerts 5. Clear alerts 6. Reset flood status 7. Exit Enter choice: 2 --- FLOOD STATUS --- BarangayA: SAFE BarangayB: SAFE BarangayC: SAFE BarangayD: SAFE BarangayE: SAFE BarangayF: SAFE </pre>	All barangays reset to SAFE.	Select option 6. Reset flood status.

Menu 7: Exit - This test ensures the program terminates gracefully when the user selects the exit option.

Actual Output	Expected Result	Steps Conducted
<pre> ===== FLOOD MONITORING SYSTEM ===== 1. Mark flooded area (Add if not exists) 2. View flood status 3. Evacuation Path recommendation 4. View emergency alerts 5. Clear alerts 6. Reset flood status 7. Exit Enter choice: 7 Exiting... </pre>	Program ends	Select option 7. Exit.

Invalid Text Cases - shows how the system handles errors and incorrect inputs (e.g., invalid barangay names, Invalid Input and , invalid menu choices), proving its error-handling robustness.

Test Case	Steps Conducted	Expected Result	Actual Output
1. Non-numeric Menu Choice	1. Run program. 2. At menu, input abc.	System rejects non-numeric input and re-prompts for valid menu choice.	<pre> ===== FLOOD MONITORING SYSTEM ===== 1. Mark flooded area (Add if not exists) 2. View flood status 3. Evacuation Path recommendation 4. View emergency alerts 5. Clear alerts 6. Reset flood status 7. Exit Enter choice: A Invalid input. Enter a number. Enter choice: </pre>
2. Invalid Barangay Name (Mark Flooded Area)	1. Select option 1. 2. Input Barangay5. 3. Answer n when prompted.	System rejects unknown barangay when user declines to add.	<pre> 7. Exit Enter choice: A Invalid input. Enter a number. Enter choice: 1 Enter area that is flooded: Barangay5 Barangay not found. Do you want to add it? (y/n): n Barangay not added. </pre>
3. Invalid Connection Barangay	1. Select option 1. 2. Input Barangay5. 3. Answer y. 4. Input Barangay77 (invalid).	System adds new barangay, rejects invalid connection, still marks area as flooded.	<pre> Enter choice: 1 Enter area that is flooded: Barangay5 Barangay not found. Do you want to add it? (y/n): y Barangay Barangay5 added successfully! Enter barangay to connect with: Barangay77 Invalid barangay to connect. !!This Barangay5 is FLOODED!! </pre>
4. Evacuation Recommendation with Invalid Barangay	System rejects evacuation recommendation for unknown barangay.	1. Select option 3. 2. Input BarangayZZ.	<pre> Enter choice: 3 Enter your barangay location: BarangayZZ Invalid area name. </pre>

5. Invalid Menu Choice	System rejects invalid menu option.	1. Run program. 2. Input 8 at menu.	<pre> ===== FLOOD MONITORING SYSTEM ===== 1. Mark flooded area (Add if not exists) 2. View flood status 3. Evacuation Path recommendation 4. View emergency alerts 5. Clear alerts 6. Reset flood status 7. Exit Enter choice: 8 Invalid choice. </pre>
-------------------------------	-------------------------------------	--	---

4.2. Performance Test (Prove that NFR1 is met by testing with the 50+ record input.)

Menu 2 with 50+ input:

```

--- FLOOD STATUS ---
BarangayA: SAFE
BarangayB: SAFE
BarangayC: SAFE
BarangayD: SAFE
BarangayE: SAFE
BarangayF: SAFE
BarangayG: SAFE
BarangayH: SAFE
BarangayI: SAFE
BarangayJ: SAFE
BarangayK: SAFE
BarangayL: SAFE
BarangayM: SAFE
BarangayN: SAFE
BarangayO: SAFE
BarangayP: SAFE
BarangayQ: SAFE
BarangayR: SAFE
BarangayS: SAFE
BarangayT: SAFE
BarangayU: SAFE
BarangayV: SAFE
BarangayW: SAFE
BarangayX: SAFE
BarangayY: SAFE
BarangayZ: SAFE
BarangayAA: SAFE
BarangayAB: SAFE
BarangayAC: SAFE
BarangayAD: SAFE
BarangayAE: SAFE
BarangayAF: SAFE
BarangayAG: SAFE
BarangayAH: SAFE
BarangayAI: SAFE
BarangayAJ: SAFE
BarangayAK: SAFE
BarangayAL: SAFE
BarangayAM: SAFE
BarangayAN: SAFE
BarangayAO: SAFE
BarangayAP: SAFE
BarangayAQ: SAFE
BarangayAR: SAFE
BarangayAS: SAFE
BarangayAT: SAFE
BarangayAU: SAFE
BarangayAV: SAFE
BarangayAW: SAFE
BarangayAX: SAFE
Barangay1: SAFE
Barangay2: SAFE
Barangay3: SAFE
BARANGAY4: SAFE
Barangay5: SAFE

```

V. Conclusion

5.1. Conclusion

The Flood Evacuation Planning System is a vivid example of how Data Structures and Algorithms (DSA) can be leveraged in the most effective way to solve the toughest problems that affect the community safety and disaster preparedness in the real world. The system, by combining vectors, queues, hash maps, sorting algorithms, graph structures, and Dijkstra's algorithm, is an efficient, reliable, and scalable solution that is in line with SDG 11: Sustainable Cities and Communities.

The project, during its development phase, was able to achieve all the planned functional requirements by giving users the ability to monitor flooded barangays, manage alerts, and calculate safe evacuation routes through shortest-path computations. Moreover, the non-functional requirements were also accomplished: the program was efficient even with large datasets (50+ records), it was able to handle invalid input smoothly, and it preserved a modular and readable code structure that is future-proof.

This system, apart from meeting the academic objectives, serves as an exemplar of how one's technical skills in C++ and algorithmic design can be of great benefit to society. By providing clear flood monitoring, giving the best evacuation options first, and organizing the management of alerts, the project is a great example of how data-driven planning is the core of disaster risk reduction. In the end, the Flood Evacuation Planning System is a concrete and effective instrument that not only deepens the resilience, but it also upholds safety and facilitates the sustainable development of the city.

5.2. Individual Contributions (Detailed breakdown of each member's assigned module/class.)

Members	Contributions
Caliguiran, Arjane	• Document: Testing and Result • Code part: Graph class and Main program
Crisolo, Angel Christian	• Document: 3.1. Core Data Structures Used (The Five) • Flowchart • Code: Input validation and Main Program
Dayag, Freddiemar	• Document: ◦ Algorithm Flowchart: ◦ Flowchart ◦ 3.3. Module Breakdown: • Code: Struct and Main program
Esteban, Maria Victoria N.	• Document: Introduction, Requirements and analysis • Code: Barangay Input file • Flowchart